Lecture 12 - Optimisation, SVMs and neural networks

COMP3670/COMP6670 - Introduction to Machine Learning

Diego Marcondes

Semester 2 2026



Australian National University

Optimisation problems

Recall Lecture 11

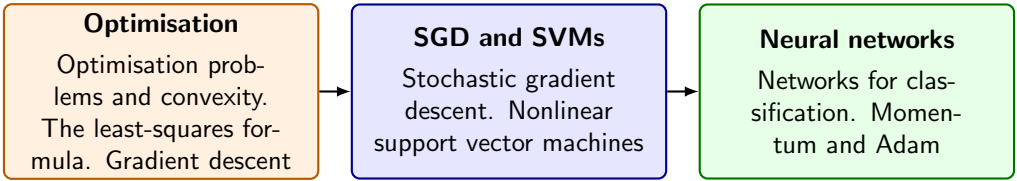
- Lecture 11 left us with two tools:
  - **The gradient** $\nabla J(\boldsymbol{\theta})$: at any point, which way is "uphill" and how steep it is
  - **Convexity**: when it holds, a point with $\nabla J(\boldsymbol{\theta}) = \mathbf{0}$ is a global minimiser
- We solved these equations analytically for three problems: the coin, the line through the origin and the line with an intercept. Each time we could *solve* the equations $\nabla J(\boldsymbol{\theta}) = \mathbf{0}$ by hand
- A real model has hundreds or millions of such equations, and this is usually not possible

The question for this lecture

How do we actually reach the point where the gradient is zero: **by a formula** or **by an algorithm**?

- Where linear algebra meets calculus

The plan for this lecture



An optimisation problem

Definition (Optimisation problem)

Given an **objective function** $J : \mathbb{R}^m \to \mathbb{R}$, an (unconstrained) **optimisation problem** asks for

$$\hat{\boldsymbol{\theta}} \in \underset{\boldsymbol{\theta} \in \mathbb{R}^m}{\arg\min}\ J(\boldsymbol{\theta}).$$

Such a $\hat{\boldsymbol{\theta}}$ is a **minimiser**, and $J(\hat{\boldsymbol{\theta}})$ is the **minimum value**.

- In learning, J is the in-sample error, or the in-sample error plus a penalty, or minus a log-likelihood, and $\boldsymbol{\theta}$ are the parameters of the model
- **Unconstrained**: every $\boldsymbol{\theta} \in \mathbb{R}^m$ is allowed. In a **constrained** problem $\boldsymbol{\theta}$ must also satisfy some conditions, like the margin requirements of the SVM

Learning is optimisation

	objective J	convex?	formula?
Least squares	$\frac{1}{N}\|\mathbf{Y} - X\boldsymbol{\theta}\|^2$	yes	yes
Ridge regression	$\|\mathbf{Y} - X\boldsymbol{\theta}\|^2 + \lambda\|\boldsymbol{\theta}\|^2$	yes	yes
Logistic regression	$-\frac{1}{N}\,\mathrm{LL}(\mathbf{w})$, the average cross-entropy	yes	no
Soft-margin SVM	$\frac{1}{2}\|\mathbf{w}\|^2 + C\sum_i \max\{0,\ 1 - y_i(\langle \mathbf{w}, \mathbf{x}_i\rangle + b)\}$	yes	no
Neural networks	an in-sample error	no	no

- Lecture 1: learning from data is *"solving the optimisation problem"* $\hat{h} = \arg\min_{h\in\mathcal{H}} E_{\mathrm{in}}(h)$. Every row is an instance of it
- Four convex problems, two with a formula and two without, and one problem that is not convex

Two ways to solve $\nabla J = \mathbf{0}$

By formula

Solve the m equations with pen and paper. Usually possible when the equations are *linear*: least squares, ridge

By iteration

Start somewhere, improve a little, and repeat until the gradient is nearly zero. More broadly applicable

- A formula is exact, but it rarely exists, and for large problems it can cost too much to evaluate
- An iteration, like gradient descent, gives an approximate answer, but for most of machine learning models it is the only option

Recognising a convex problem

- Nobody computes eigenvalues to check convexity. Convex objectives are **assembled** from convex pieces:

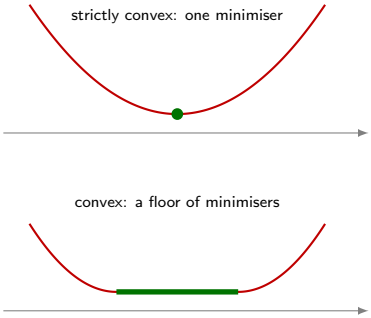
Rules that preserve convexity

(1) Sums and positive multiples of convex functions. **(2)** A convex function of a linear map, such as $\boldsymbol{\theta} \mapsto X\boldsymbol{\theta} - \mathbf{Y}$. **(3)** The maximum of convex functions.

objective	why it is convex
Least squares	the squared norm, a convex function, of the linear map $\boldsymbol{\theta} \mapsto \mathbf{Y} - X\boldsymbol{\theta}$ (rule 2)
Ridge regression	least squares plus the convex penalty $\lambda\|\boldsymbol{\theta}\|^2$ (rule 1)
Logistic regression	a sum of terms $-\log\sigma(\cdot)$, each convex in a score that is linear in $\mathbf{w}$ (rules 1, 2)
Soft-margin SVM	the hinge is a maximum of two straight lines (rule 3), of a linear margin (rule 2), plus $\frac{1}{2}\|\mathbf{w}\|^2$

Convex functions

- **Every critical point is a global minimiser**. The equation $\nabla J(\boldsymbol{\theta}) = \mathbf{0}$ is not a list of candidates, it is the answer
- **Every local minimum is global**. An algorithm that keeps going down cannot be trapped
- **The starting point does not matter** for where we end. It only affects how long it takes
- **Strictly convex**, for instance, Hessian positive definite: the minimiser is unique. Merely convex: there may be a flat valley floor of minimisers, all equally good



Least squares in matrix form

- The linear model of Lecture 4 is $Y = \theta_0\phi_0(\mathbf{X}) + \cdots + \theta_k\phi_k(\mathbf{X}) + \varepsilon$, with basis functions ϕ_j. Stack the data: $\mathbf{Y} = (Y_1, \ldots, Y_N)^T$ and the $N \times (k+1)$ **design matrix**

$$X = \begin{pmatrix} \phi_0(\mathbf{X}_1) & \cdots & \phi_k(\mathbf{X}_1) \\ \vdots & & \vdots \\ \phi_0(\mathbf{X}_N) & \cdots & \phi_k(\mathbf{X}_N) \end{pmatrix}, \qquad \mathrm{RSS}(\boldsymbol{\theta}) = \|\mathbf{Y} - X\boldsymbol{\theta}\|^2$$

- One **row** per observation and one **column** per parameter. Denote by $m = k+1$ the number of columns. Row i of $X\boldsymbol{\theta}$ is the prediction for observation i, and $\mathbf{Y} - X\boldsymbol{\theta}$ is the vector of residuals
- Running example, with $\phi_0 = 1$ and $\phi_1(x) = x$: the rows of X are $(1,1)$, $(1,2)$, $(1,3)$, $(1,4)$ and $\mathbf{Y} = (2,5,4,9)^T$

Reference: Braga-Neto (2024), *Fundamentals of Pattern Recognition and Machine Learning*, Section 11.3.1, equations (11.20)–(11.25).

Setting the gradient to zero: the normal equations

- Least squares is convex, so a critical point is all we need. From Lecture 11, $\nabla \mathrm{RSS}(\boldsymbol{\theta}) = -2X^T(\mathbf{Y} - X\boldsymbol{\theta})$. Set it to zero:

The normal equations

$$-2X^T(\mathbf{Y} - X\boldsymbol{\theta}) = 0 \implies 2X^T\mathbf{Y} = 2X^TX\boldsymbol{\theta} \implies X^TX\,\boldsymbol{\theta} = X^T\mathbf{Y}$$

- A linear system of m **linear** equations in m unknowns. If X has full column rank, X^TX is invertible and the solution is unique
- **The geometry**. The equations say $X^T(\mathbf{Y} - X\hat{\boldsymbol{\theta}}) = \mathbf{0}$ so the residual is orthogonal to every column of X. So $X\hat{\boldsymbol{\theta}}$ is the **orthogonal projection** of $\mathbf{Y}$ onto the column space of X
- Running example: $\begin{pmatrix} 4 & 10 \\ 10 & 30 \end{pmatrix} \boldsymbol{\theta} = \begin{pmatrix} 20 \\ 60 \end{pmatrix}$ has the solution $\hat{\boldsymbol{\theta}} = (0, 2)^T$

Reference: Braga-Neto (2024), *Fundamentals of Pattern Recognition and Machine Learning*, Section 11.3.1, equation (11.26) and Exercise 11.3.

The least-squares solution

- Because the model is linear in $\boldsymbol{\theta}$, the normal equations can be solved in **closed form**: if X has full column rank,

$$X^T X\,\boldsymbol{\theta} \;=\; X^T\mathbf{Y} \;\Longrightarrow\; \hat{\boldsymbol{\theta}} \;=\; \left(X^T X\right)^{-1} X^T\mathbf{Y}, \qquad\qquad \hat{h}(\mathbf{x}) = \boldsymbol{\Phi}(\mathbf{x})^T\hat{\boldsymbol{\theta}}$$

- **No algorithm is needed**. Linear models are one case where ERM is solved exactly
- For the simple line $h_{\boldsymbol{\theta}}(x) = \theta_0 + \theta_1 x$ it reduces to the formulas we derived by hand in Lecture 11:

$$\hat{\theta}_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}, \qquad \hat{\theta}_0 = \bar{y} - \hat{\theta}_1 \bar{x}$$

Reference: Braga-Neto (2024), *Fundamentals of Pattern Recognition and Machine Learning*, Section 11.3.1, equations (11.23)–(11.29).

Penalised least squares

- High-order models overfit (recall polynomial regression). One alternative is to choose the order by validation. Another is to **keep the order but stop the coefficients becoming large**

Ridge regression

$$\hat{\boldsymbol{\theta}}^{\text{RIDGE}} = \arg\min_{\boldsymbol{\theta}} \underbrace{\|\mathbf{Y} - X\boldsymbol{\theta}\|^2}_{\text{fit}} + \underbrace{\lambda\|\boldsymbol{\theta}\|^2}_{\text{penalty}}, \qquad \lambda > 0$$

$$\hat{\boldsymbol{\theta}}^{\text{RIDGE}} = \left(X^T X + \lambda I\right)^{-1} X^T \mathbf{Y}$$

- The criterion is **no longer** E_{in}: it is E_{in} plus a term with nothing to do with the data, but with our preference for simplicity
- Compare with the soft-margin SVM: *hinge* + $\|\mathbf{w}\|^2$ there, *squared* + $\lambda\|\boldsymbol{\theta}\|^2$ here
- $\lambda = 0$ is ordinary least squares, large λ forces the coefficients towards zero (**shrinkage**)

Reference: Braga-Neto (2024), *Fundamentals of Pattern Recognition and Machine Learning*, Section 11.3.3, equations (11.44)–(11.45).

Why the penalty also helps the computation

- Where does the ridge formula come from? The criterion is $J(\boldsymbol{\theta}) = \|\mathbf{Y} - X\boldsymbol{\theta}\|^2 + \lambda\|\boldsymbol{\theta}\|^2$, and the table of gradients gives $\nabla J(\boldsymbol{\theta}) = -2\,X^T(\mathbf{Y} - X\boldsymbol{\theta}) + 2\lambda\,\boldsymbol{\theta}$. Setting it to zero:

$$-2\,X^T(\mathbf{Y}-X\boldsymbol{\theta})+2\lambda\,\boldsymbol{\theta} = 0 \implies 2\big(X^TX+\lambda I\big)\,\boldsymbol{\theta}-2X^T\mathbf{Y} = 0 \implies \big(X^TX+\lambda I\big)\,\boldsymbol{\theta} = X^T\mathbf{Y}$$

- The Hessian is $2\big(X^TX + \lambda I\big)$. Adding λI increases **every eigenvalue** of X^TX by λ, so they are at least $\lambda > 0$

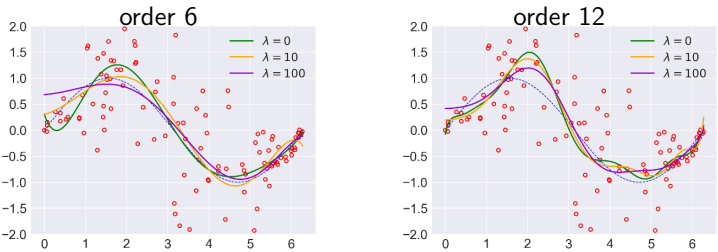
Penalty and convexity

With $\lambda > 0$ the ridge criterion is **strictly convex**: the minimiser exists and is unique, and $X^TX + \lambda I$ is **always invertible**, even when X does not have full column rank (more parameters than observations, or duplicated features).

- Regularisation makes the learning problem *well-posed*

Reference: Braga-Neto (2024), *Fundamentals of Pattern Recognition and Machine Learning*, Section 11.3.3.

Shrinkage example



- High-order polynomials fitted with three values of λ
- Larger λ gives visibly **flatter, smoother** curves
- So $\hat{h}$ is **the best fit in the chosen basis, subject to a budget on the coefficients**: we deliberately *accept more bias to get less variance*
- λ tunes the complexity **continuously**, instead of jumping from one degree to the next

Source: Braga-Neto (2024), Figure 11.4 (Example 11.6).

The gradient of the cross-entropy

- Logistic regression: the probability of class 1 is $\sigma(\mathbf{w}^T\mathbf{X})$, with a 1 in front of the features so that the bias is one of the weights. The objective is $J(\mathbf{w}) = \frac{1}{N}\sum_i \ell_i(\mathbf{w})$, with

$$\ell_i(\mathbf{w}) \;=\; -\,Y_i \log \sigma(\mathbf{w}^T\mathbf{X}_i) \;-\; (1-Y_i)\log\left(1-\sigma(\mathbf{w}^T\mathbf{X}_i)\right)$$

- By the chain rule, with $\sigma' = \sigma(1-\sigma)$, we have

The gradient of the cross-entropy

$$\nabla\ell_i(\mathbf{w}) \;=\; \left(\sigma(\mathbf{w}^T\mathbf{X}_i) - Y_i\right)\mathbf{X}_i, \qquad \nabla J(\mathbf{w}) \;=\; \frac{1}{N}\sum_{i=1}^{N}\left(\sigma(\mathbf{w}^T\mathbf{X}_i) - Y_i\right)\mathbf{X}_i$$

- **Gradient is error times the input**: the predicted probability minus the label, times the features. And the objective is convex, so a point where this gradient vanishes is the global minimiser

Reference: Braga-Neto (2024), *Fundamentals of Pattern Recognition and Machine Learning*, Section 4.3, equations (4.31) and (4.32).

No analytical solution

- **No formula at all**. For logistic regression, $\nabla J(\mathbf{w}) = \mathbf{0}$ reads $\sum_i \left(\sigma(\mathbf{w}^T \mathbf{X}_i) - Y_i\right)\mathbf{X}_i = \mathbf{0}$. The unknown $\mathbf{w}$ sits *inside* σ: the equations are **not linear**, and no algebra will isolate $\mathbf{w}$

So we iterate

When the equations $\nabla J(\mathbf{w}) = \mathbf{0}$ cannot be solved, we can apply **gradient descent**.

Gradient descent

The gradient descent algorithm

Gradient descent

1. **Initialise**: choose a starting point $\boldsymbol{\theta}(0)$
2. For each iteration $t = 0, 1, 2, \ldots$: compute the gradient at the current point and **update**

$$\boldsymbol{\theta}(t+1) \leftarrow \boldsymbol{\theta}(t) - \alpha \nabla J\big(\boldsymbol{\theta}(t)\big)$$

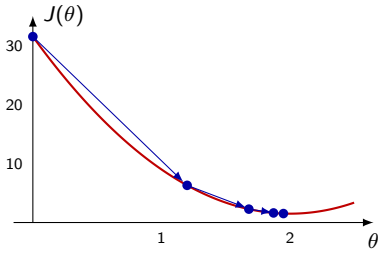
3. **Repeat** step 2 until a stopping rule is met

- $\alpha > 0$ is the **learning rate**, or step size: how far we move in the downhill direction. It is a **hyperparameter**, not learnt from the data

Numerical example: gradient descent by hand

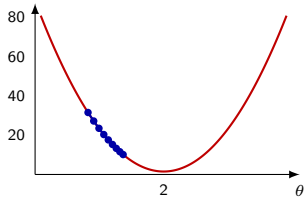
- The one-parameter running example: $J(\theta) = 7.5\,\theta^2 - 30\,\theta + 31.5$ and $J'(\theta) = 15\,\theta - 30$. Take $\alpha = 0.04$ and start at $\theta(0) = 0$, so that $\theta(t+1) = \theta(t) - 0.04\,\big(15\,\theta(t) - 30\big)$

t	$\theta(t)$	$J'\big(\theta(t)\big)$	$J\big(\theta(t)\big)$
0	0	-30	31.5
1	1.2	-12	6.3
2	1.68	-4.8	2.27
3	1.872	-1.92	1.62
4	1.949	-0.77	1.52

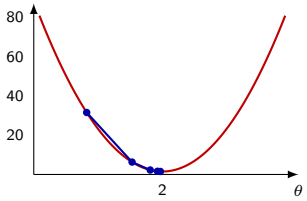


- The steps shrink as the slope flattens: 1.2, then 0.48, 0.19, 0.08
- Each step multiplies the distance to $\hat{\theta} = 2$ by $1 - 15\,\alpha = 0.4$. We never arrive, but we get **as close as we like**: after 10 steps the distance is 0.0002

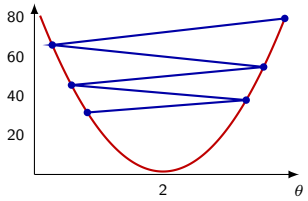
The learning rate



$\alpha = 0.005$: too small



$\alpha = 0.04$: about right



$\alpha = 0.14$: too large

- Each step multiplies the distance to the minimiser by $1 - 15\,\alpha$. For a small α the factor is close to 1: after 8 steps we have barely moved. Beyond $\alpha = 2/15 \approx 0.133$ it is below -1: every step overshoots further, and the iterates **diverge**
- The curvature J'' is 15, and **the greater it is, the smaller the learning rate must be**
- The learning rate tunes the *algorithm*, not the model, since the minimum being sought remains the same

Theoretical guarantees of gradient descent

Theorem (Gradient descent on a convex objective)

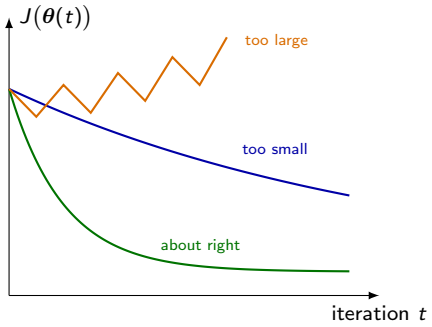
Let J be convex and well-behaved, with curvature at most K in every direction and at every point (no eigenvalue of the Hessian exceeds K). If the learning rate satisfies $\alpha \leq 1/K$, then from **any** *starting point $\boldsymbol{\theta}(0)$,*

$$J\big(\boldsymbol{\theta}(t)\big) - J(\hat{\boldsymbol{\theta}}) \;\leq\; \frac{\|\boldsymbol{\theta}(0) - \hat{\boldsymbol{\theta}}\|^2}{2\,\alpha\,t}.$$

- We state it **without proof**.
- It means that after t steps, the objective is within a constant over t of its minimum. Ten times more steps, the bound is ten times smaller
- Both hypotheses matter: without convexity nothing of the kind is true, and with α too large we have just seen the iterates diverge

When to stop

- The iteration never arrives at the minimum, so we must decide when to stop. Four rules, usually combined:
  - **the gradient is small**: we are nearly at a critical point
  - **the objective has stalled**: no decrease for a while
  - **the validation error has stopped improving**
  - **set a budget**: a maximum number of iterations
- Plotting the objective against the iteration number shows whether the learning rate is too small, too large or about right



Gradient descent for our three problems

	one step of gradient descent
Least squares, $\frac{1}{N}\|\mathbf{Y} - X\boldsymbol{\theta}\|^2$	$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \frac{2\alpha}{N} X^T(\mathbf{Y} - X\boldsymbol{\theta})$
Ridge, $\|\mathbf{Y} - X\boldsymbol{\theta}\|^2 + \lambda\|\boldsymbol{\theta}\|^2$	$\boldsymbol{\theta} \leftarrow (1 - 2\alpha\lambda)\,\boldsymbol{\theta} + 2\alpha X^T(\mathbf{Y} - X\boldsymbol{\theta})$
Logistic regression	$\mathbf{w} \leftarrow \mathbf{w} - \frac{\alpha}{N}\sum_i \left(\sigma(\mathbf{w}^T\mathbf{X}_i) - Y_i\right)\mathbf{X}_i$

- **Current parameters, plus a correction built from error times input**
- Ridge first shrinks the parameters by the factor $1 - 2\alpha\lambda$, and then corrects them

The cost of one step

- Every gradient of the last slides is a sum over **all** the observations:

$$\nabla J(\boldsymbol{\theta}) \;=\; \frac{1}{N}\sum_{i=1}^{N} \nabla \ell_i(\boldsymbol{\theta})$$

- One step of gradient descent requires the gradient of the loss evaluated at every data point. Its computational cost is proportional to $N \times m$, the number of observations times the number of parameters
- The running example has $N = 4$. The datasets used to train modern models have millions, or billions, of points, together with millions, or billions, of parameters
- And gradient descent needs hundreds or thousands of steps!

The computational bottleneck

The gradient is an **average over the whole data set**. Computing that average exactly makes every step expensive.

Stochastic gradient descent

A statistician's idea

- Pick one index i uniformly at random from $\{1, \ldots, N\}$, and use $\mathbf{g} = \nabla \ell_i(\boldsymbol{\theta})$ in place of the full gradient. Its expectation is

$$\mathbb{E}[\mathbf{g}] \;=\; \sum_{i=1}^{N} \underbrace{\frac{1}{N}}_{\mathbb{P}(\mathbf{g} = \nabla \ell_i(\boldsymbol{\theta}))} \underbrace{\nabla \ell_i(\boldsymbol{\theta})}_{\text{value of } \mathbf{g}} \;=\; \nabla J(\boldsymbol{\theta})$$

- $\mathbf{g}$ is an **unbiased** estimate of the gradient, so it is right on average
- Better: draw a **mini-batch** B of b indices at random and average, $\mathbf{g} = \frac{1}{b} \sum_{i \in B} \nabla \ell_i(\boldsymbol{\theta})$. It is still unbiased and, like any sample mean, it fluctuates less as b grows
- $b = N$ is gradient descent, while $b = 1$ is the cheapest and the noisiest

The trade

A cheap, noisy gradient in place of an expensive, exact one. For the price of one exact gradient we can afford N/b noisy ones, and take N/b steps instead of one.

Stochastic gradient descent

Stochastic gradient descent (SGD)

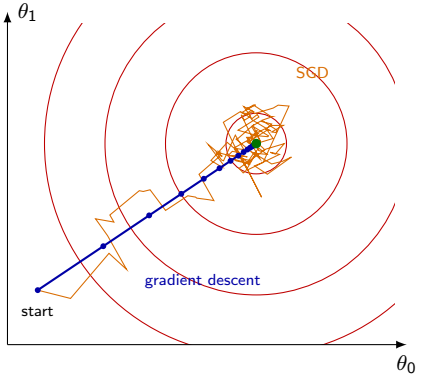
1. **Initialise**: choose a starting point $\boldsymbol{\theta}(0)$ and a batch size b
2. For each iteration $t = 0, 1, 2, \ldots$: draw a mini-batch B_t of b observations at random, compute

$$\mathbf{g}(t) \;=\; \frac{1}{b} \sum_{i \in B_t} \nabla \ell_i\big(\boldsymbol{\theta}(t)\big) \qquad \text{and } \textbf{update} \qquad \boldsymbol{\theta}(t+1) \;\leftarrow\; \boldsymbol{\theta}(t) - \alpha_t\, \mathbf{g}(t)$$

3. **Repeat** step 2 until a stopping rule is met

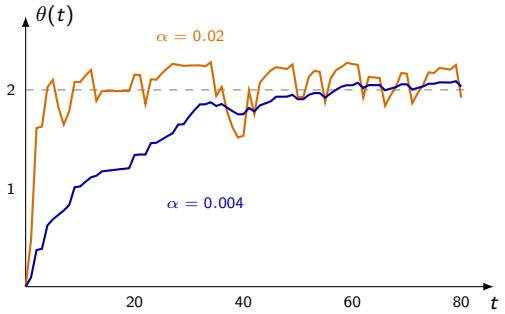
- **Batch size**: commonly between 32 and 512
- **Epoch**: one pass through the data, N/b iterations. In practice the data is shuffled and broken into consecutive mini-batches
- The learning rate now carries an index, α_t, as it may change during training

What the path looks like



- A least-squares problem with $N = 100$ observations, two parameters
- **Gradient descent**: 15 steps, each one using all 100 observations. A total of 1500 gradients
- **SGD** with $b = 1$: 150 steps, each one using a single observation. A total of 150 gradients, a tenth of the computations
- SGD wanders around, but it reaches the neighbourhood of the minimiser at a fraction of the cost
- And then it does not settle

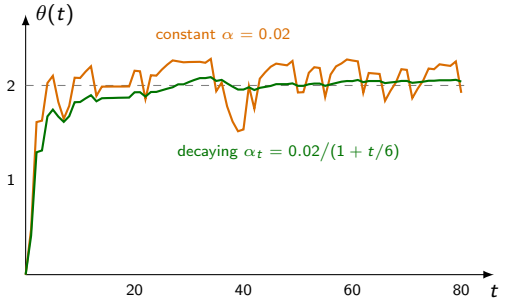
The price of noise



- The one-parameter running example, with mini-batches of one observation
- Left alone, each observation would choose its own slope: $y_i/x_i = 2,\ 2.5,\ 1.33,\ 2.25$
- At $\hat{\theta} = 2$ the full gradient is zero, but the four individual gradients might not. They only **cancel on average**

- Each SGD step follows one of them, and is pulled towards that observation's own answer. So with a constant learning rate SGD **never settles**: it wanders around the minimiser
- A larger α arrives sooner and wanders more. A smaller α wanders less and arrives later

Learning-rate schedules



- **The fix**: large steps at the start, to arrive quickly, and small steps at the end, to settle. A rule for α_t is a **learning-rate schedule**
- In the picture: the same random observations, in the same order

- Common schedules: **piecewise constant**, dividing α by 10 at chosen epochs, and **exponential**, $\alpha_t = \alpha_0 \, 0.1^{\,t/s}$ for some $s > 0$

Why SGD is the default

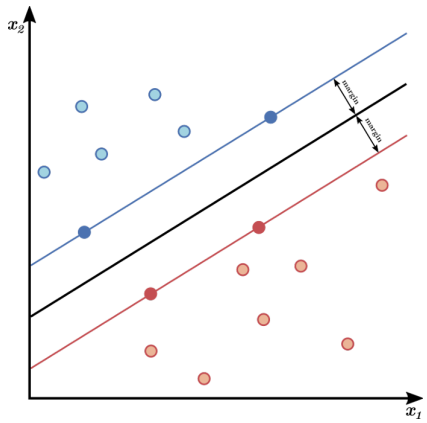
- **The cost of a step does not depend on N.** A data set ten times larger costs nothing more per step
- **Fast early progress.** In the time gradient descent takes one step, SGD takes N/b. Far from the minimiser, even a noisy direction might be a good direction
- **A minimiser is not exactly what we want.** E_{in} is itself a noisy estimate of E_{out}, so its minimiser is only meaningful up to the generalisation gap. Ten digits of accuracy are meaningless
- **The noise can help.** On non-convex problems it nudges the iterates out of poor local minima and off saddles

The default of machine learning

Almost every model you will see in practice is trained by SGD or one of its descendants, and "gradient descent" nearly always means the stochastic version.

Nonlinear SVMs

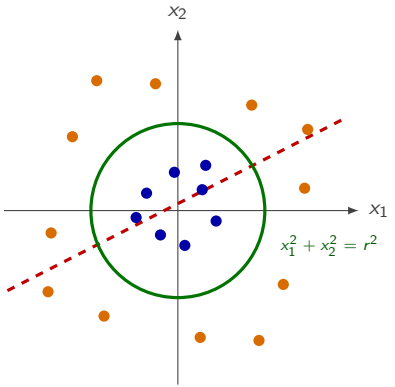
Recall SVM



Source: Braga-Neto (2024), Figure 6.1.

- Labels $y_i \in \{-1, +1\}$. A linear classifier computes the **score** $f(\mathbf{x}) = \langle \mathbf{w}, \mathbf{x} \rangle + b$ and predicts $+1$ when $f(\mathbf{x}) \geq 0$, and -1 otherwise
- The **functional margin** $m_i = y_i f(\mathbf{x}_i)$ is positive exactly when observation i is correctly classified. Its distance to the boundary is $|f(\mathbf{x}_i)|/\|\mathbf{w}\|$
- Separable data has infinitely many separating hyperplanes. The perceptron returns whichever it reaches first, the SVM returns the one **farthest from the data**
- The closest observations, the solid points, are the **support vectors**. The boundary depends on them alone: move a point anywhere outside the margin and nothing changes

A straight line is not always enough



The dashed line is one attempt among infinitely many: whichever line we draw, one side has points of both colours.

- The perceptron, logistic regression and the linear SVM all choose among **hyperplanes**. For this data, *no* hyperplane separates the two classes
- Yet the classes are not mixed at all: the rule **"+1 when** $x_1^2 + x_2^2 < r^2$**"** classifies every point correctly
- That rule is built from the same features, **squared**. It is **linear** in the quantities x_1^2 and x_2^2, and curved in x_1 and x_2

Nonlinear SVMs: change the variables

- **The idea**: keep the linear classifier, and **change the variables**. Choose a transformation ϕ sending each $\mathbf{x}$ to a new vector, replace every $\mathbf{x}_i$ by $\phi(\mathbf{x}_i)$, and run the **ordinary linear SVM** on the transformed data:

$$J(\mathbf{w}, b) \;=\; \frac{1}{2}\|\mathbf{w}\|^2 \;+\; C\sum_{i=1}^{N} \max\left\{0,\; 1 - y_i\left(\langle \mathbf{w}, \phi(\mathbf{x}_i)\rangle + b\right)\right\}$$

- Nothing in the algorithm changes. The transformation is fixed before training, so the objective is **still convex** in $(\mathbf{w}, b)$, and the update of Week 8b applies with $\phi(\mathbf{x}_i)$ in place of $\mathbf{x}_i$
- A hyperplane in the new variables, interpreted in the old ones, is a **curved boundary**

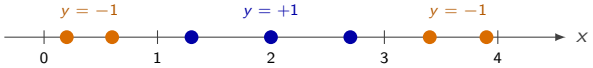
Reference: Braga-Neto (2024), *Fundamentals of Pattern Recognition and Machine Learning*, Section 6.1.3.

Example: one feature

- Take $d = 1$, so each observation is a single number x, and suppose the true rule is

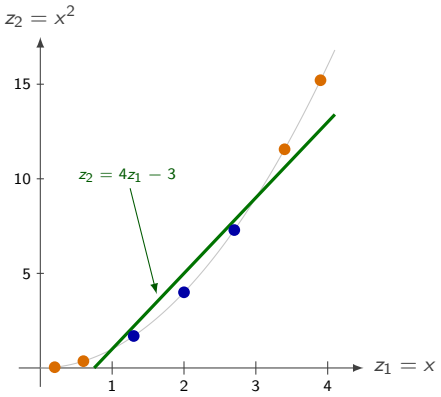
$$f^\star(x) \;=\; +1 \quad \text{exactly when} \quad 1 < x < 3, \qquad f^\star(x) = -1 \quad \text{otherwise}$$

- A linear classifier on one feature is $h(x) = \text{sign}(w\,x + b)$: it is $+1$ on **one side of a single threshold**. It can never be $+1$ on a middle interval and -1 on *both* sides



- **The data is not linearly separable**

Add one variable, and it separates



- Send each number x to the **pair** $\phi(x) = (x, x^2)$. One feature became two, and the points now lie on a parabola
- In the new variables a **straight line** separates them: the blue points fall below $z_2 = 4z_1 - 3$ and the orange ones above
- Interpret that line back in terms of x. It says $x^2 < 4x - 3$, that is

$$x^2 - 4x + 3 < 0 \quad \Longleftrightarrow \quad (x-1)(x-3) < 0$$

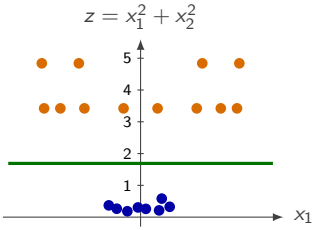
which holds exactly when $1 < x < 3$: **the rule we started from**

A linear classifier in (x, x^2) is a *quadratic* classifier in x. The straight line in two dimensions is the pair of thresholds in one dimension.

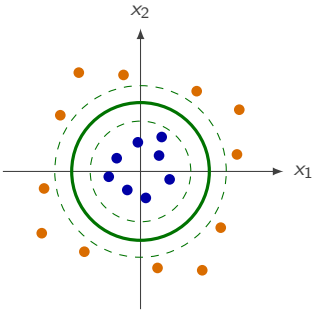
Back to the circle in $d = 2$

- Send $\mathbf{x} = (x_1, x_2)$ to **two** numbers,

$$\phi(\mathbf{x}) \;=\; \left(x_1,\; x_1^2 + x_2^2\right)$$



The transformed data: the line $z = r^2$ separates the two classes.

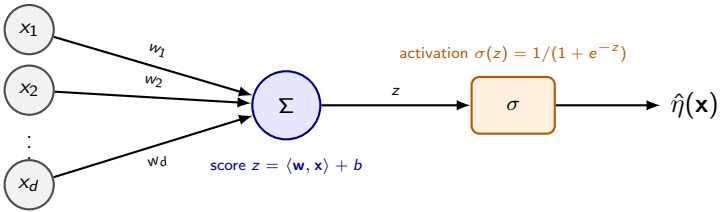


The same line in the original variables: a *circle*.

- Same algorithm, same criterion. **Only the vector of variables changed**, and the straight boundary became a round one

Neural networks

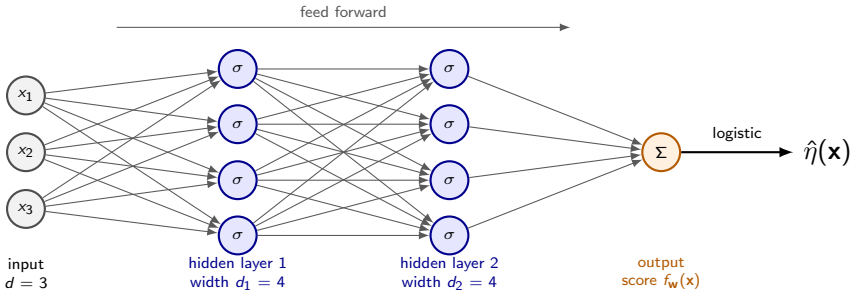
Logistic regression is a single neuron



- A **neuron** computes a weighted sum of its inputs plus a bias, the score, and applies a fixed nonlinear function to it, the **activation**
- Logistic regression is **one neuron** with the sigmoid activation. Its output estimates $\eta(\mathbf{x}) = \mathbb{P}(Y = 1 \mid \mathbf{X} = \mathbf{x})$, and its decision boundary is a **hyperplane**
- For curved boundaries the SVM changed the variables, with a transformation chosen in advance. A **neural network** places layers of neurons between the input and the output, and **learns** the new variables from the data

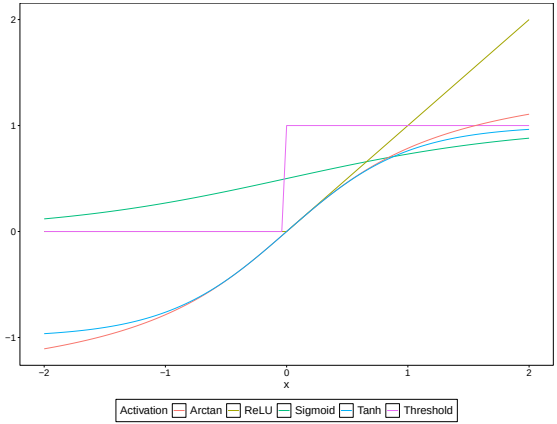
Reference: Braga-Neto (2024), *Fundamentals of Pattern Recognition and Machine Learning*, Sections 6.2.1 and 6.2.2.

Fully connected deep neural networks (FCDNN)



- Every blue circle is a neuron: a weighted sum of **all** the outputs of the previous layer, plus a bias, followed by an activation σ. Hence the name **fully connected**
- $k = 2$ **hidden layers** of **width** 4; the **depth**, counting the input and output layers, is $k + 2 = 4$. Information flows only from left to right: the network is **feed forward**
- The output is a score $f_{\mathbf{w}}(\mathbf{x})$, turned into $\hat{\eta}(\mathbf{x})$ by the logistic function: the logistic regression of the previous slide, on the outputs of the last hidden layer

Activation functions



$\arctan(z)$, $\quad \mathrm{ReLU}(z) = \max\{0, z\}$, $\quad$ sigmoid $1/(1+e^{-z})$, $\quad \tanh(z)$, $\quad$ threshold $\mathbb{1}\{z \geq 0\}$

Neural networks in matrix notation

- Write the weights of layer i as a matrix A_i, with entries $(A_i)_{l,j} = w_{l,j}^{(i)}$. Then for the network drawn earlier, with σ applied coordinate by coordinate:

The network in matrix notation

$$f_{\mathbf{w}}(\mathbf{x}) \;=\; A_3\,\sigma\big(A_2\,\sigma(A_1\,\mathbf{x})\big)$$

matrix	size	linear map	between
A_1	$d_1 \times d = 4 \times 3$	$\mathbb{R}^3 \to \mathbb{R}^4$	the input and hidden layer 1
A_2	$d_2 \times d_1 = 4 \times 4$	$\mathbb{R}^4 \to \mathbb{R}^4$	hidden layers 1 and 2
A_3	$1 \times d_2 = 1 \times 4$	$\mathbb{R}^4 \to \mathbb{R}$	hidden layer 2 and the output

- In general, $f_{\mathbf{w}}(\mathbf{x}) = A_{k+1}\,\sigma\big(A_k \cdots \sigma(A_1\,\mathbf{x})\big)$, with A_i of size $d_i \times d_{i-1}$, the number of neurons in the hidden layers, and A_{k+1} of size $1 \times d_k$ ($K \times d_k$ for K classes). The weights $\mathbf{w}$ are the entries of all the matrices
- A bias adds one column to A_i, which multiplies the coordinate equal to 1 added to its input

The output layer for classification

- In regression the output $f_{\mathbf{w}}(\mathbf{x})$ is the prediction. In classification the network must output **probabilities**, so the last layer is followed by a different activation
- **Two classes**, $Y \in \{0, 1\}$: the logistic function turns the output into an estimate of $\eta(\mathbf{x}) = \mathbb{P}(Y = 1 \mid \mathbf{X} = \mathbf{x})$, and the plug-in rule compares it with $\frac{1}{2}$:

$$\hat{\eta}_{\mathbf{w}}(\mathbf{x}) \;=\; \frac{1}{1 + e^{-f_{\mathbf{w}}(\mathbf{x})}}, \qquad\qquad \hat{h}(\mathbf{x}) \;=\; \mathbb{1}\{\hat{\eta}_{\mathbf{w}}(\mathbf{x}) > \tfrac{1}{2}\} \;=\; \mathbb{1}\{f_{\mathbf{w}}(\mathbf{x}) > 0\}$$

- K **classes**: the last layer has K outputs $f_{\mathbf{w},1}(\mathbf{x}), \ldots, f_{\mathbf{w},K}(\mathbf{x})$, and the **softmax** turns them into K positive probabilities that add up to 1:

$$\hat{p}_{\mathbf{w},k}(\mathbf{x}) \;=\; \frac{e^{f_{\mathbf{w},k}(\mathbf{x})}}{\sum_{j=1}^{K} e^{f_{\mathbf{w},j}(\mathbf{x})}}, \qquad\qquad \hat{h}(\mathbf{x}) \;=\; \arg\max_{k} \; \hat{p}_{\mathbf{w},k}(\mathbf{x})$$

- With no hidden layer and two classes, the network is exactly **logistic regression**

Reference: Braga-Neto (2024), *Fundamentals of Pattern Recognition and Machine Learning*, Section 6.2.2.

Universal approximation of neural networks

Theorem (Cybenko, 1989)

Let f be a continuous function defined on the hypercube $[0,1]^d$. For any $\varepsilon > 0$, there is a one-hidden-layer neural network

$$\zeta(\mathbf{x}) = \sum_{i=0}^{d_1} w_i^{(2)} \sigma\Big(\sum_{j=0}^{d} w_{i,j}^{(1)} x_j \Big),$$

with $x_0 = 1$ and sigmoid activation, such that $|f(\mathbf{x}) - \zeta(\mathbf{x})| < \varepsilon$ for all $\mathbf{x} \in [0,1]^d$.

Reference: Braga-Neto (2024), *Fundamentals of Pattern Recognition and Machine Learning*, Section 6.2.7.

Training a neural network for classification

- Training data $(\mathbf{X}_1, Y_1), \ldots, (\mathbf{X}_N, Y_N)$, with labels $Y_i \in \{0, 1\}$
- Empirical **cross-entropy** loss, as for logistic regression:

$$J(\mathbf{w}) \;=\; -\frac{1}{N}\sum_{i=1}^{N}\Big[Y_i \log \hat{\eta}_{\mathbf{w}}(\mathbf{X}_i) + (1 - Y_i)\log\big(1 - \hat{\eta}_{\mathbf{w}}(\mathbf{X}_i)\big)\Big]$$

- With K classes, $J(\mathbf{w}) = -\frac{1}{N}\sum_i \log \hat{p}_{\mathbf{w}, Y_i}(\mathbf{X}_i)$: minus the log of the probability given to the correct class
- Empirical Risk Minimisation:

$$\begin{aligned}&\text{minimise } J(\mathbf{w})\\ &\text{subject to } \mathbf{w} \in \mathbb{R}^p, \text{ p is the number of parameters}\end{aligned}$$

- **Non-convex** optimisation problem with multiple local minima

Reference: Braga-Neto (2024), *Fundamentals of Pattern Recognition and Machine Learning*, Section 6.2.3.

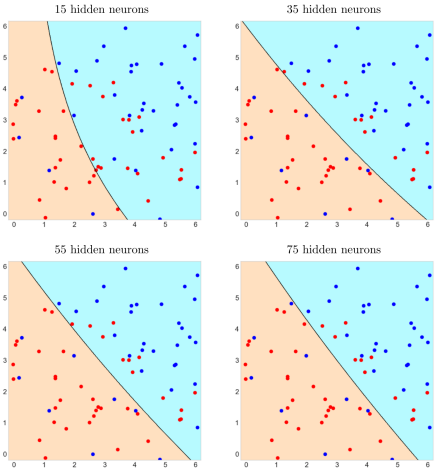
Neural networks overfit classification data

- For mild width and depth, a FCDNN represents a very complex class of functions
- Therefore, it may perfectly separate small samples, without properly capturing the relationship between $\mathbf{X}$ and Y
- Complexity regularisation, such as minimising

$$J(\mathbf{w}) + \lambda \|\mathbf{w}\|^2,$$

  is usually considered to avoid overfitting
- Unless a perfect separation of the data is desired, for instance when the label is a deterministic function of $\mathbf{X}$

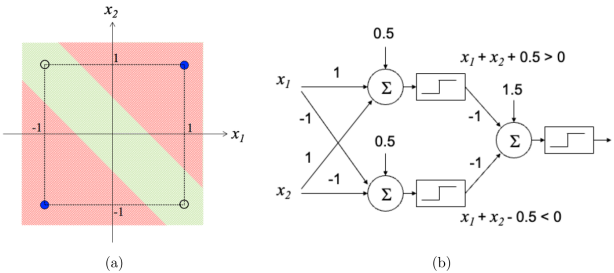
Neural network classification example



Source: Braga-Neto (2024), Figure 6.8 (Example 6.4).

- Two classes with Gaussian distributions. The optimal boundary is a line with slope $-45°$ through the centre of the plot
- One hidden layer with a varying number of neurons, one output neuron, sigmoid activations, trained on the cross-entropy loss by gradient descent (1 000 iterations)
- With 15 neurons the network **underfits**. With 35 neurons the boundary is close to the optimal one
- More neurons do not improve the result: it becomes slightly worse, a small degree of **overfitting**

Neural network classification example: XOR



- The XOR data: no line separates the two classes, so a single perceptron cannot classify them. A network with **one hidden layer of two neurons** can, with step activations and weights chosen by hand
- Each hidden neuron is a perceptron implementing one of the two parallel lines $x_1+x_2+0.5 = 0$ and $x_1 + x_2 - 0.5 = 0$. The output neuron, another perceptron, combines them into the labels of the regions

Source: Braga-Neto (2024), Figure 6.7 (Example 6.3)

Gradient descent algorithm

- The most common approach to minimise the loss $J(\mathbf{w})$ is gradient descent
- One starts with a randomly initialised weight vector $\mathbf{w}(0) \in \mathbb{R}^p$, and performs the basic step

$$\mathbf{w}(t+1) \;=\; \mathbf{w}(t) + \alpha\, \mathbf{v}(t)$$

  in which $\alpha > 0$ is the **learning rate** and $\mathbf{v}(t)$ is a direction vector
- The greedy step in the direction of maximum local decrease of J is

$$\mathbf{v}(t) \;=\; -\nabla J\big(\mathbf{w}(t)\big),$$

  which characterises the standard **gradient descent** method
- The gradient is computed by the chain rule, applied layer by layer from the output backwards (**backpropagation**)

Reference: Braga-Neto (2024), *Fundamentals of Pattern Recognition and Machine Learning*, Sections 6.2.4 and 6.2.5.

Initialisation

- Neural networks are trained with variants of the stochastic gradient descent algorithm
- Properly choosing the initial value $\mathbf{w}(0)$ is **critical** to finding a good local minimum
- Heuristic approaches, in which each weight is initialised by sampling from a zero-mean uniform or Gaussian distribution
- For the weights connecting a layer of d_i neurons to the next layer, of d_{i+1} neurons, the variance is usually scaled as a function of d_i and d_{i+1}
- There are a number of variants:
  - "LeCun" initialisation: variance $1/d_i$
  - "Xavier" or "Glorot" initialisation: variance $2/(d_i + d_{i+1})$
  - "He" initialisation: variance $2/d_i$

Reference: Braga-Neto (2024), *Fundamentals of Pattern Recognition and Machine Learning*, Section 6.2.5.

Momentum and Adam

Heavy ball method

- Gradient descent can take too big steps in regions of high curvature of the loss (creating oscillation), or too small steps in flat regions
- The heavy-ball algorithm counters it by introducing **momentum**
- *"A heavy ball does not bounce so much off walls, and also rolls faster on flat surfaces"*
- The heavy ball update is

$$\mathbf{m}(t) = \beta\,\mathbf{m}(t-1) - \alpha\,\nabla J\big(\mathbf{w}(t)\big)$$
$$\mathbf{w}(t+1) = \mathbf{w}(t) + \mathbf{m}(t)$$

  where $\mathbf{m}$ is the momentum vector with $\mathbf{m}(0) = 0$, and $0 < \beta < 1$ is an exponential decay parameter

- A value $\beta = 0.9$ is common

Reference: Braga-Neto (2024), *Fundamentals of Pattern Recognition and Machine Learning*, Section 6.2.5, equation (6.71).

Nesterov accelerated gradient

- The Nesterov method is a small variation of the heavy ball method, which usually accelerates convergence, by computing the gradient not at the current value of $\mathbf{w}$, but at a "future" one
- The intuition is that the gradient at a point closer to the minimum is more desirable, as it should point more towards the minimum
- The Nesterov update is

$$\begin{aligned}\mathbf{m}(t) &= \beta\,\mathbf{m}(t-1) - \alpha\,\nabla J\big(\mathbf{w}(t) + \beta\,\mathbf{m}(t-1)\big)\\ \mathbf{w}(t+1) &= \mathbf{w}(t) + \mathbf{m}(t)\end{aligned}$$

  where $0 < \beta < 1$ as before

- Again, $\beta = 0.9$ is a common choice

Reference: Braga-Neto (2024), *Fundamentals of Pattern Recognition and Machine Learning*, Section 6.2.5.

Adagrad

- Adagrad (adaptive gradient descent) is a method to adjust the learning rate of each coordinate
- One would like the learning rate to be smaller along the directions of steepest descent, and larger in the directions where there is little gradient
- Adagrad accomplishes this with the update

$$\mathbf{s}(t) = \mathbf{s}(t-1) + \Big(\nabla J\big(\mathbf{w}(t)\big)\Big)^2$$

$$\mathbf{w}(t+1) = \mathbf{w}(t) - \frac{\alpha}{\sqrt{\mathbf{s}(t) + \epsilon}} \nabla J\big(\mathbf{w}(t)\big)$$

with $\mathbf{s}(0) = 0$ where $\epsilon > 0$ is a small number to avoid division by zero, and the vector operations are coordinate-wise

Reference: Braga-Neto (2024), *Fundamentals of Pattern Recognition and Machine Learning*, Section 6.2.5.

RMSProp

- RMSProp is a popular variation of Adagrad, which introduces exponential decay, so that older gradients are "forgotten"
- The RMSProp update is

$$\mathbf{s}(t) = \beta\,\mathbf{s}(t-1) + (1-\beta)\Big(\nabla J\big(\mathbf{w}(t)\big)\Big)^2$$

$$\mathbf{w}(t+1) = \mathbf{w}(t) - \frac{\alpha}{\sqrt{\mathbf{s}(t)+\epsilon}}\,\nabla J\big(\mathbf{w}(t)\big)$$

where $0 < \beta < 1$ is the exponential decay parameter, usually set to $\beta = 0.9$

Reference: Braga-Neto (2024), *Fundamentals of Pattern Recognition and Machine Learning*, Section 6.2.5.

Adaptive moment estimation (Adam)

- Finally, Adam is a hybrid between the momentum-based and the Adagrad-based classes of methods. The Adam update is

$$\begin{aligned}
\mathbf{m}(t) &= \beta_1\,\mathbf{m}(t-1) - (1-\beta_1)\,\nabla J\big(\mathbf{w}(t)\big) \\
\mathbf{s}(t) &= \beta_2\,\mathbf{s}(t-1) + (1-\beta_2)\Big(\nabla J\big(\mathbf{w}(t)\big)\Big)^2 \\
\widehat{\mathbf{m}}(t) &= \mathbf{m}(t)\big/\big(1-\beta_1^t\big) \\
\widehat{\mathbf{s}}(t) &= \mathbf{s}(t)\big/\big(1-\beta_2^t\big) \\
\mathbf{w}(t+1) &= \mathbf{w}(t) + \frac{\alpha}{\sqrt{\widehat{\mathbf{s}}(t)+\epsilon_{\text{root}}}+\epsilon}\,\widehat{\mathbf{m}}(t)
\end{aligned}$$

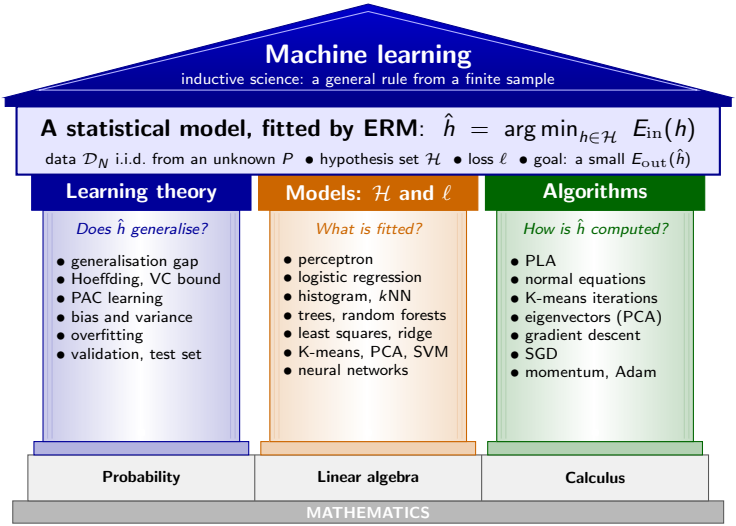
- Here $0 < \beta_i < 1$, $i = 1, 2$, and ϵ_{root}, ϵ are small numbers. The usual choices are $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$ and $\epsilon_{\text{root}} = 0$
- **Adam is the default method for training neural networks**

Reference: Braga-Neto (2024), *Fundamentals of Pattern Recognition and Machine Learning*, Section 6.2.5, equation (6.75).

Lecture highlights

- **Optimisation problem**: training a model is minimising an in-sample error, possibly penalised
- **Convex problems**: every local minimum is global
- **Normal equations**: solution $\hat{\boldsymbol{\theta}} = \left(X^T X\right)^{-1} X^T \mathbf{Y}$ for least squares
- **Gradient descent**: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \nabla J(\boldsymbol{\theta})$, learning rate, convergence, stopping rules
- **Stochastic gradient descent**: mini-batches, epochs, learning-rate schedules
- **Nonlinear SVMs**: support vectors, change of variables, curved boundaries from a linear classifier
- **Neural networks**: fully connected networks, activation functions, depth and width, output layer and softmax for classification, cross-entropy, universal approximation, overfitting, initialisation
- **Momentum and Adam**: heavy ball, Nesterov, Adagrad, RMSProp, Adam

Introduction to Machine Learning



The main message

Machine learning is **not** a collection of algorithms. It is **inductive science**: a **statistical model** is fitted to data by **empirical risk minimisation**, and **statistical learning theory** guarantees that what is learned generalises.

The **algorithms**, built on linear algebra and calculus, are the tools to train the model, not the model itself.